Flood Detection of SAR Images Using UNETR

Project: GenAI Remote Sensing

Mentor: Devika Ma'am

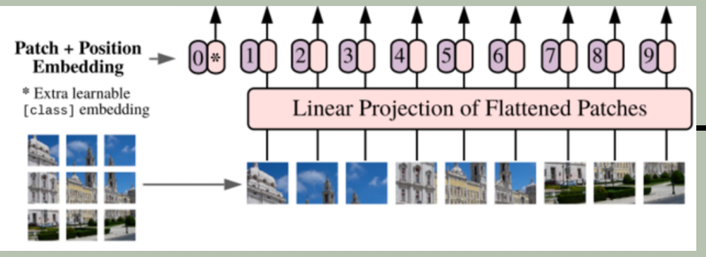
WHAT IS UNETR?

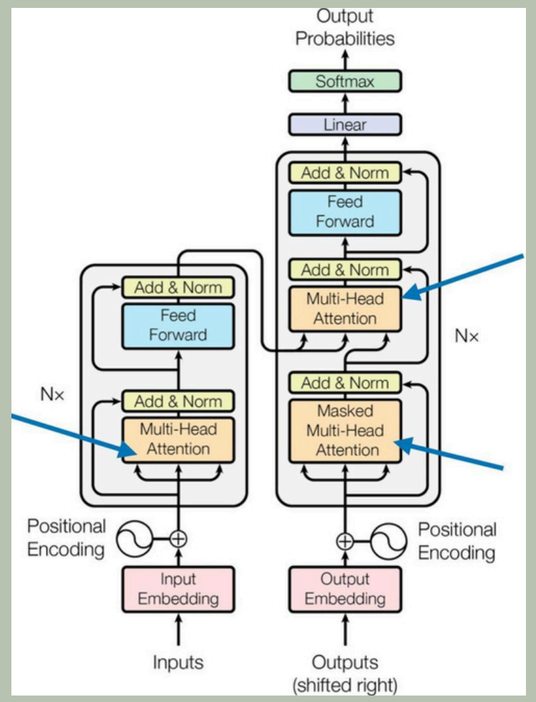
- UNETR (U-Net with Transformer Encoder) is an advanced neural network architecture for semantic segmentation tasks.
- Combines transformer encoders with U-Net for semantic segmentation.
- Captures global and local features simultaneously.

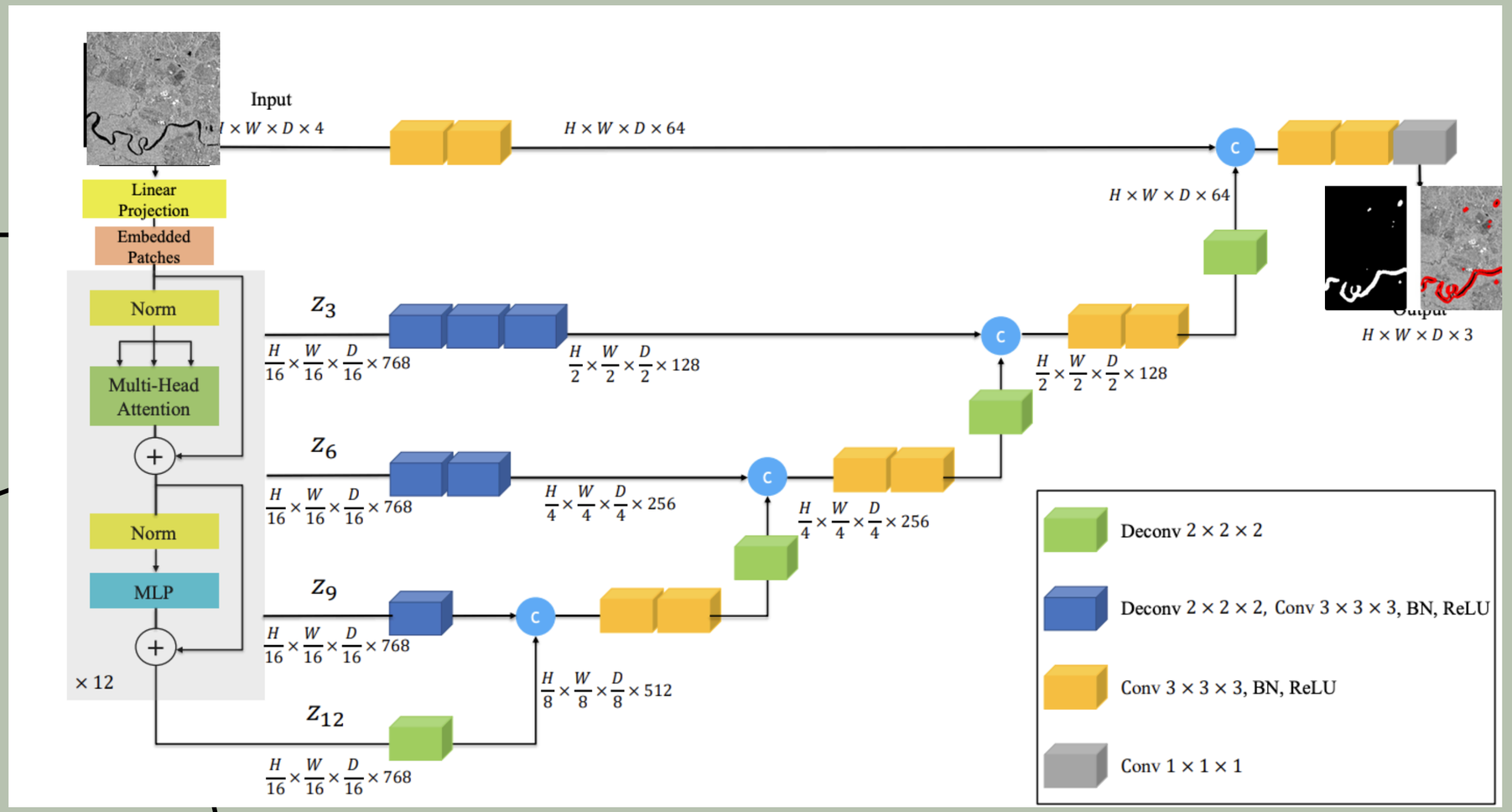
WHY USE UNETR FOR FLOOD DETECTION?

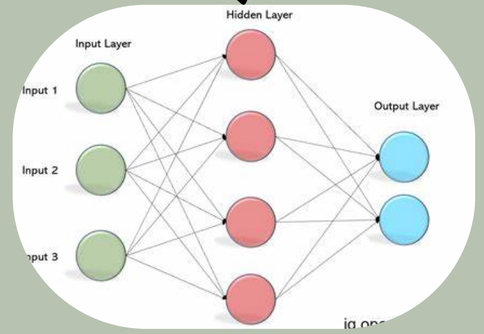
- Accurate segmentation: Identifies flooded vs. non-flooded areas pixel by pixel.
- Global understanding: Handles irregular flood patterns using transformers.
- SAR compatibility: Works well with noisy VV-polarized SAR images.
- Scalable: Ideal for high-resolution satellite imagery.

UNETR ARCHITECTURE

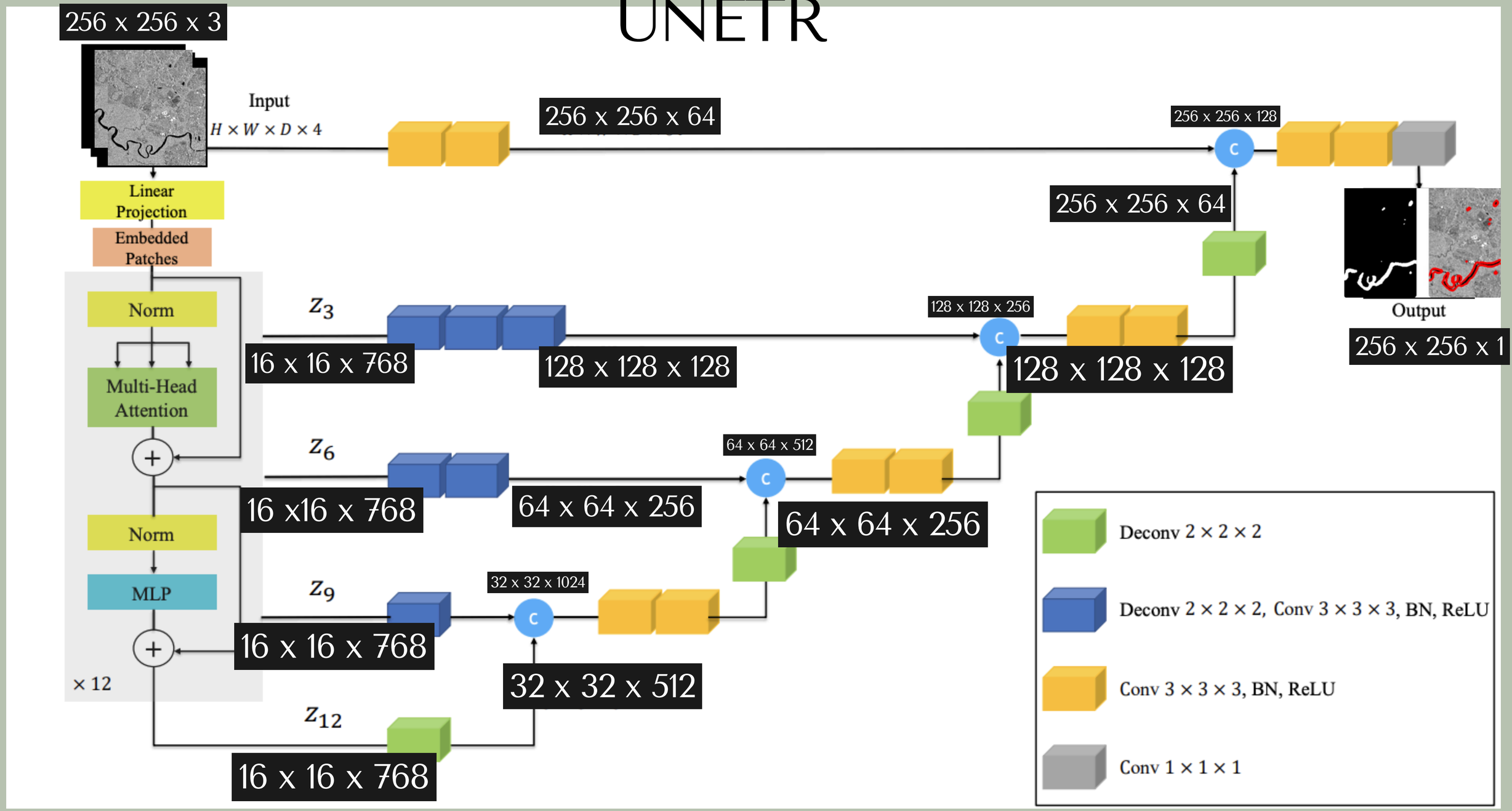






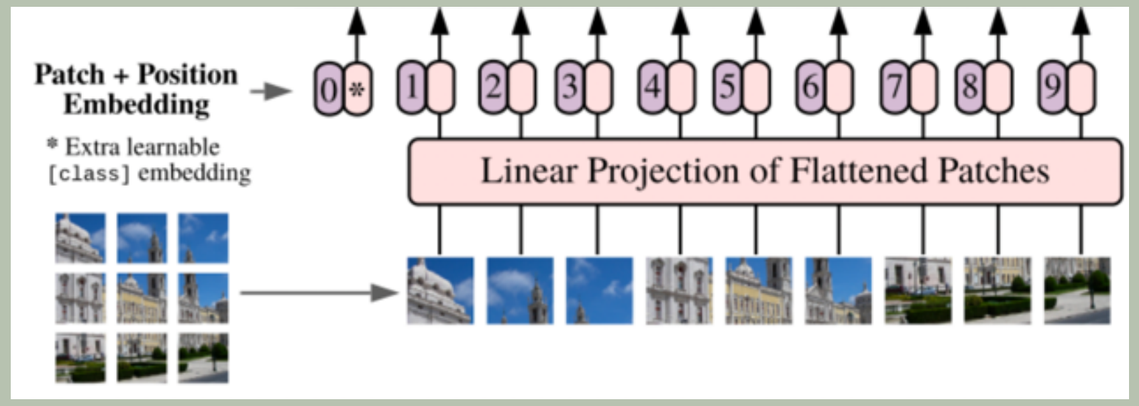


UNETR


256 x 256 x 3
Input
$H \times W \times D \times 4$
Linear Projection
Embedded Patches
Norm
Multi-Head Attention
Norm
MLP
× 12
z_3
16 x 16 x 768
128 x 128 x 128
z_6
16 x16 x 768
64 x 64 x 256
z_9
16 x 16 x 768
32 x 32 x 1024
32 x 32 x 512
z_{12}
16 x 16 x 768
256 x 256 x 64
256 x 256 x 128
256 x 256 x 64
128 x 128 x 256
128 x 128 x 128
64 x 64 x 512
64 x 64 x 256
Output
256 x 256 x 1
Deconv 2 × 2 × 2
Deconv 2 × 2 × 2, Conv 3 × 3 × 3, BN, ReLU
Conv 3 × 3 × 3, BN, ReLU
Conv 1 × 1 × 1

Linear Projection

```
Input Shape: (256, 768)

1. Input Size and Dimensions:
Input Shape: (256, 768)
Input Values Sample (first patch): Cannot show numpy values
```



UNETR

```
if __name__ == "__main__":
    config = {
        "image_size": 256,
        "num_layers": 12,
        "hidden_dim": 128,
        "mlp_dim": 32,
        "num_heads": 6,
        "dropout_rate": 0.1,
        "patch_size": 16,
        "num_patches": (256**2) // (16**2),
        "num_channels": 3,
    }
```

Mathematical Process Behind Linear Projection

1. Input Patch Representation:
    - Suppose a patch from an image is represented as a 3D array of pixel values:
      Patch∈R^H×W×C where: H=Height, W=Weight, C=Channels (3 for RGB)
    - For a 16×16 patch, the total number of pixel values = 16×16×3=768.
2. Flatten the Patch:
    - Convert the 3D array into a 1D vector: x∈R^768
3. Weight Matrix:
    - A learnable weight matrix W∈R^(768×128) is initialized during training.
    - This matrix defines how the input patch data is transformed into a smaller feature vector of size 128.
4. Dot Product (Linear Transformation):
    - Perform a matrix multiplication between the input vector x and the weight matrix W: y=x⋅W
    - y∈R^128 is the resulting feature vector.
        - Each value in y is a weighted combination of the original 768 pixel values.

```
Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_14>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_17>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_20>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_22>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_25>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_28>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_31>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_34>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_36>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_39>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_42>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_45>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_48>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_50>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_53>

Skip Connection at Layer 3:
Skip Connection Shape: (1, 256, 128)
Skip Connection Values (first few): Cannot show numpy values
```

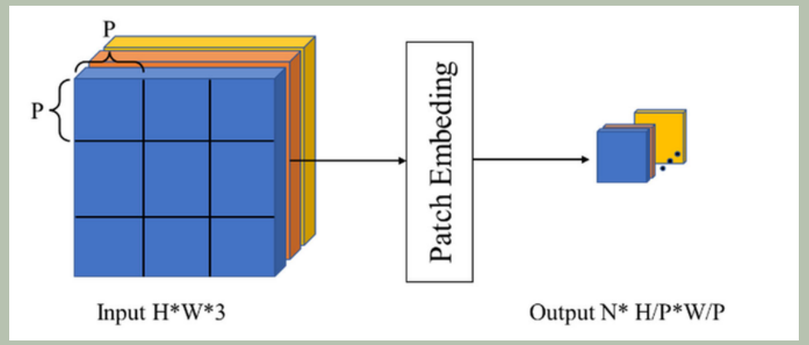
```
5. Skip Connection Outputs:
z3 Shape: (1, 256, 128)
z6 Shape: (1, 256, 128)
z9 Shape: (1, 256, 128)
z12 Shape: (1, 256, 128)
```

```
Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_56>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_59>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_62>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_64>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_67>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_70>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_73>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_76>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_78>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_81>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_84>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_87>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_90>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_92>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_95>

Skip Connection at Layer 6:
Skip Connection Shape: (1, 256, 128)
Skip Connection Values (first few): Cannot show numpy values
```

```
Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_98>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_101>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_104>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_106>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_109>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_112>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_115>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_118>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_120>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_123>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_126>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_129>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_132>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_134>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_137>

Skip Connection at Layer 9:
Skip Connection Shape: (1, 256, 128)
Skip Connection Values (first few): Cannot show numpy values
```

```
Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_140>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_143>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_146>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_148>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_151>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_154>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_157>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_160>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_162>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_165>

Transformer Encoder Input Shape: (1, 256, 128)
Transformer Encoder Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_168>
Multi-Head Attention Output Shape: (1, 256, 128)
Multi-Head Attention Output Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_171>
MLP Input Shape: (1, 256, 128)
MLP Input Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_174>
MLP Dense Layer Output Shape (cf['mlp_dim']): (1, 256, 32)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 32), dtype=float32, sparse=False, name=keras_tensor_176>
MLP Dense Layer Output Shape (cf['hidden_dim']): (1, 256, 128)
MLP Dense Layer Values (first few): <KerasTensor shape=(10, 128), dtype=float32, sparse=False, name=keras_tensor_179>

Skip Connection at Layer 12:
Skip Connection Shape: (1, 256, 128)
Skip Connection Values (first few): Cannot show numpy values
```

EMBEDDING PATCHES



3D PATCH EMBEDDING (STEP 3)

```
4. Patch + Positional Embedding:
Combined Embedding Shape: (1, 256, 128)
Combined Embedding Values (first few): Cannot show numpy values

Transformer Encoder Input Shape: (1, 256, 128)
```

PATCH EMBEDDING (STEP 1)

```
2. Patch Embedding:
Patch Embedding Shape: (None, 256, 128)
Patch Embedding Values (first few): Cannot show numpy values
```

POSITIONAL EMBEDDING (STEP 2)

```
3. Positional Embedding:
Positional Embedding Shape: (1, 256, 128)
Positional Embedding Values (first few): [[-0.02055498  0.0308024   0.0236449  ...  0.01970444  0.04956976
  -0.03483075]
 [ 0.00939238 -0.0145579   0.02572102 ... -0.0258157   0.01709776
  -0.03247984]
 [-0.0486418  -0.00420947  0.01861611 ... -0.02393901 -0.04808674
  -0.0019709 ]
 ...
 [ 0.04873333  0.04006708  0.0350534  ...  0.04100383  0.0437489
   0.01746721]
 [ 0.00533915 -0.02413813 -0.03363176 ...  0.03530386  0.01897648
   0.007085  ]
 [ 0.02084564  0.04121116 -0.03920386 ...  0.00693329 -0.02591171
  -0.02055624]]
```

Normalization

In the UNETR architecture, normalization plays a crucial role in stabilizing and accelerating the training process. The paper mentions the use of Layer Normalization within the transformer encoder and Batch Normalization in the convolutional layers of the decoder.

NORMALIZATION TECHNIQUES:

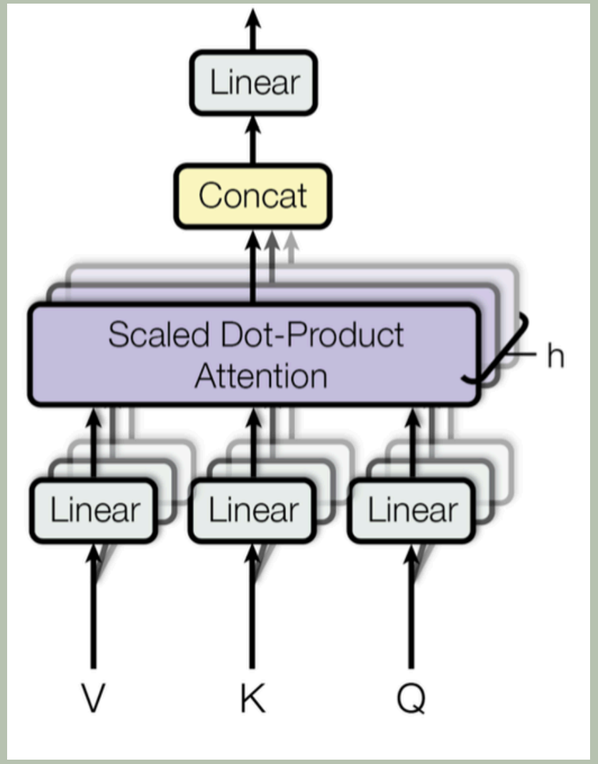
Layer Normalization: Applied within the transformer encoder to stabilize training by normalizing inputs across features.

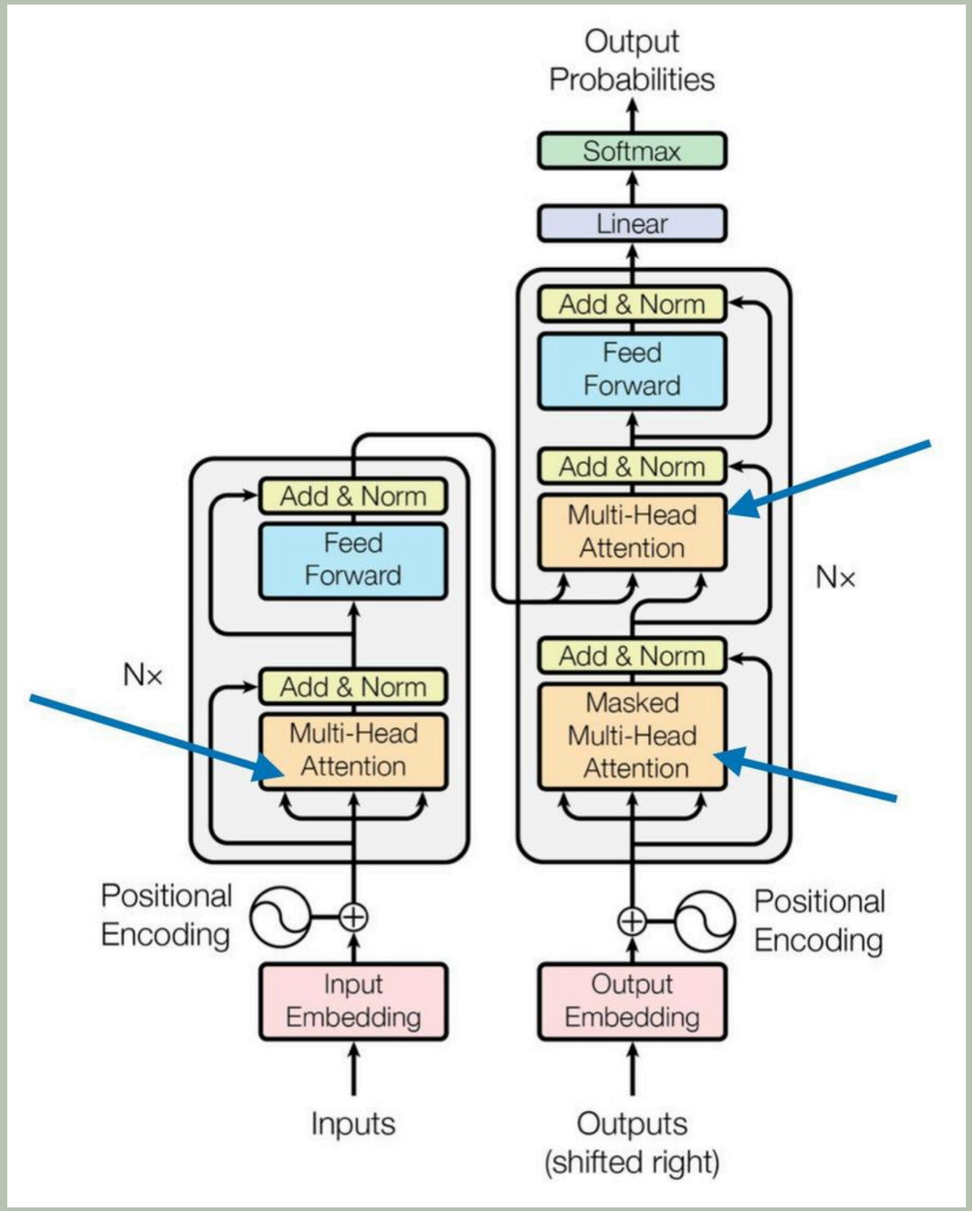
- Formula: $\text{Norm}(z_i) = (z_i - \mu) / (\sigma + \varepsilon)$
- Normalizes inputs across features.
- Helps maintain network stability during training.

Batch Normalization: Used in the convolutional layers of the decoder to reduce internal covariate shift and improve convergence.

- Formula: $\text{BN}(x) = (x - E[x]) / \sqrt{(\text{Var}[x] + \varepsilon)}$
- Normalizes inputs across the batch dimension.
- Reduces internal covariate shift and improves network convergence.
- Role in UNETR:
- Ensures effective training and performance.
- Helps the network learn both global and local features efficiently.

MULTI HEADED ATTENTION

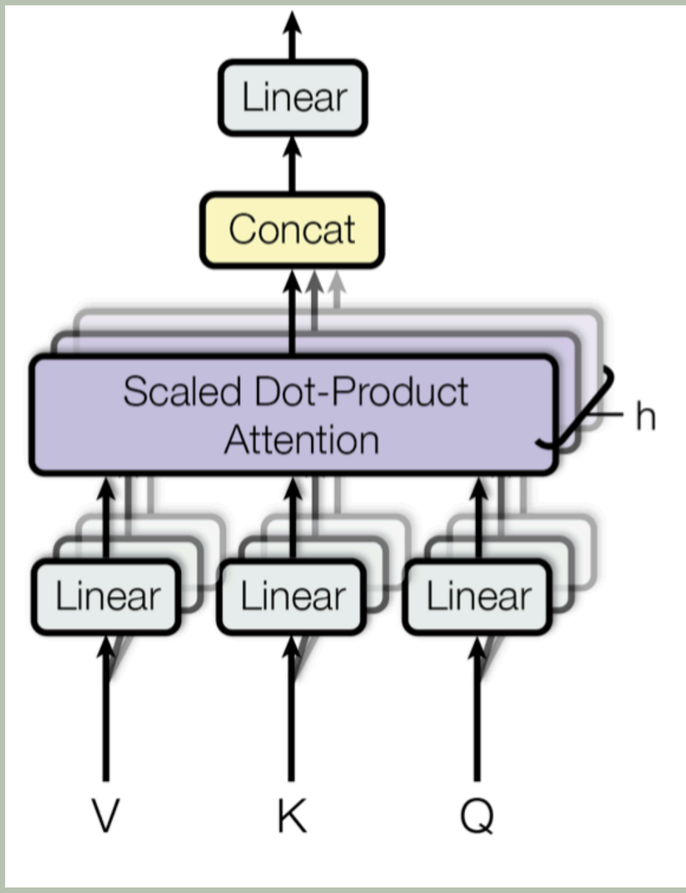




WHAT IS ATTENTION?

Attention is a mechanism used in machine learning, particularly in deep learning models, to determine the relative importance of different parts of the input data. It allows the model to focus on specific parts of the input when making predictions, rather than treating all parts equally.

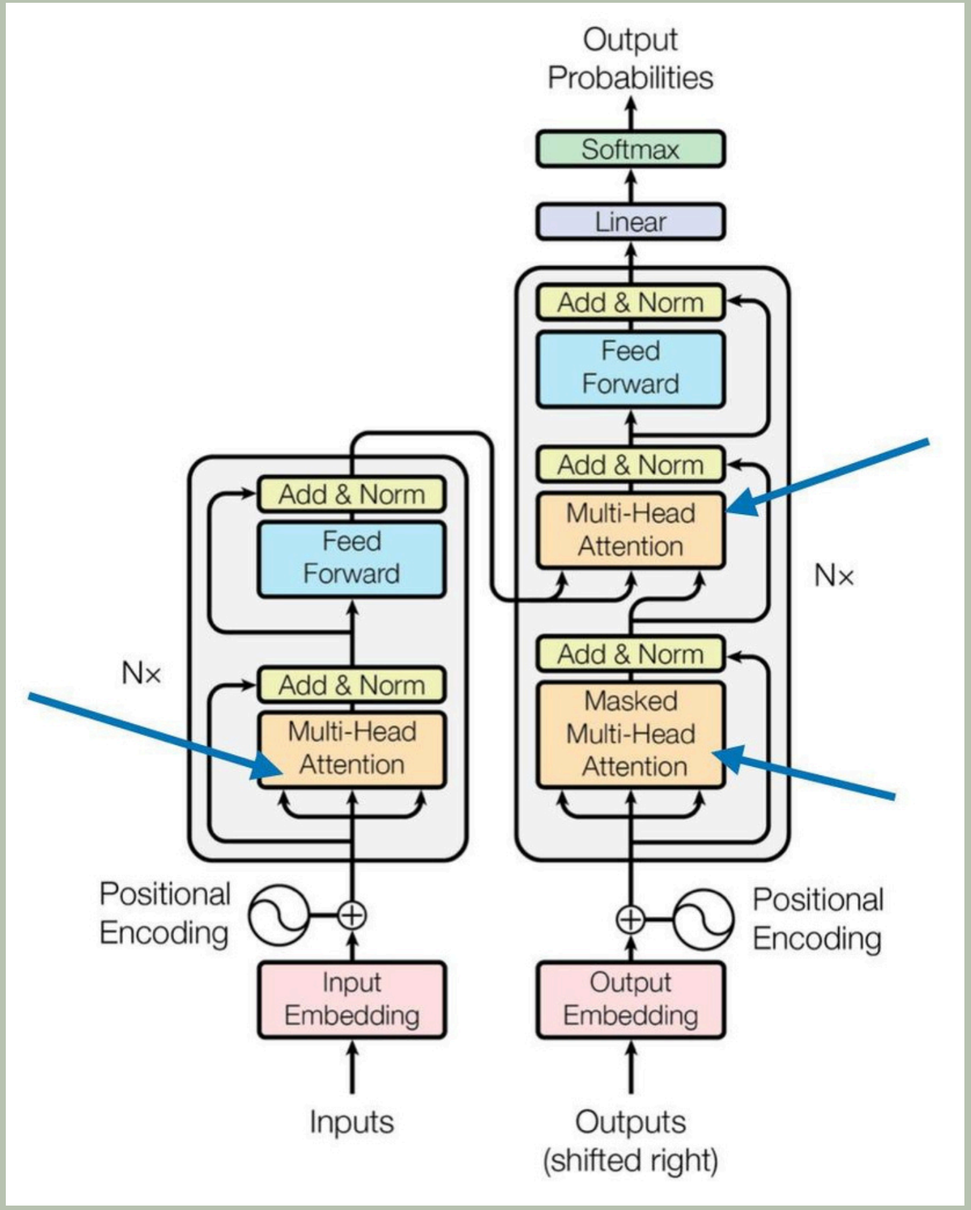
SCALED DOT-PRODUCT ATTENTION



1. Query (Q) : This represents the search criteria.
2. Key (K) : This is the reference or index used to match the query.
3. Value (V) : This is the information or answer being searched for.

Steps :

1. Dot Product Calculation : Compute the dot product between the Query and Key to measure similarity. This helps in determining how relevant each key is to the query.
   - Attention Score = Q * K^T
2. Scaling : To prevent large values causing issues during training, divide the attention scores by the square root of the dimension of the keys (dk).
   - Scaled Score = (Q * K^T) / sqrt(dk)
3. Softmax : Apply the softmax function to these scaled scores to convert them into probabilities. This step normalizes the scores so they sum up to 1 and can be interpreted as attention weights.
   - Attention Weights = softmax((Q * K^T) / sqrt(dk))
4. Weighted Sum : Multiply these attention weights by the corresponding Value vectors to compute the final output. This step aggregates the values based on their relevance.
   - Output = Attention Weights * V



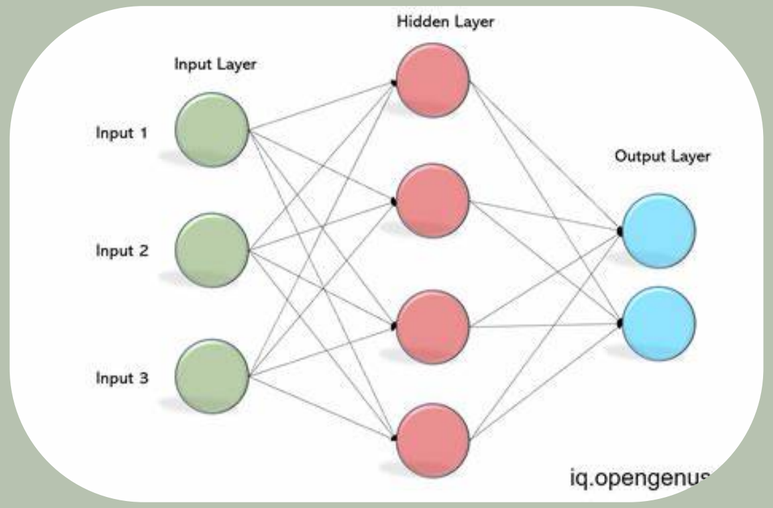
1. **Input Embedding :** This box represents the initial embedding of the input tokens into a continuous vector space.
2. **Positional Encoding :** This box adds positional information to the input embeddings to account for the order of the tokens.
3. **Nx (left side) :** This represents the encoder stack, which is repeated Nx times. Each encoder layer consists of:
   - Multi-Head Attention : This layer allows the model to focus on different parts of the input sequence simultaneously.
   - Add & Norm : This layer adds the input of the Multi-Head Attention layer to its output and normalizes the result.
   - Feed Forward : This layer is a fully connected feed-forward network applied to each position separately.
   - Add & Norm : This layer adds the input of the Feed Forward layer to its output and normalizes the result.
4. **Nx (right side) :** This represents the decoder stack, which is repeated Nx times. Each decoder layer consists of:
   - Masked Multi-Head Attention : This layer is similar to Multi-Head Attention but masks future positions to prevent the model from peeking at future tokens.
   - Add & Norm : This layer adds the input of the Masked Multi-Head Attention layer to its output and normalizes the result.
   - Multi-Head Attention : This layer allows the decoder to focus on different parts of the encoder's output.
   - Add & Norm : This layer adds the input of the Multi-Head Attention layer to its output and normalizes the result.
   - Feed Forward : This layer is a fully connected feed-forward network applied to each position separately.
   - Add & Norm : This layer adds the input of the Feed Forward layer to its output and normalizes the result.
5. **Output Embedding :** This box represents the initial embedding of the output tokens into a continuous vector space.
6. **Positional Encoding :** This box adds positional information to the output embeddings to account for the order of the tokens.
7. **Linear :** This layer projects the output of the decoder stack to the vocabulary size.
8. **Softmax :** This layer converts the logits from the Linear layer into probabilities over the vocabulary.
9. **Output Probabilities :** This represents the final output probabilities for each token in the vocabulary.

Multi-Layer Perceptron

A Multilayer Perceptron (MLP) is a feedforward neural network with fully connected neurons and non-linear activation functions, commonly used for tasks where data is not linearly separable, such as image recognition, natural language processing, and speech recognition.

Key Layers:

- Input Layer:
  - Consists of neurons that receive input data, each representing a feature or dimension.
  - The number of neurons corresponds to the input data dimensionality.
- Hidden Layer(s):
  - Layers between input and output layers.
  - Neurons in hidden layers receive input from the previous layer and pass output to the next.
  - Number of layers and neurons are hyperparameters.
- Output Layer:
  - Neurons produce the final output.
  - In classification tasks, the number of neurons depends on the task: one for binary classification, multiple for multi-class classification.



APPLICATIONS:

MLPs are a fundamental part of deep learning, capable of approximating any function, making them versatile for various domains.

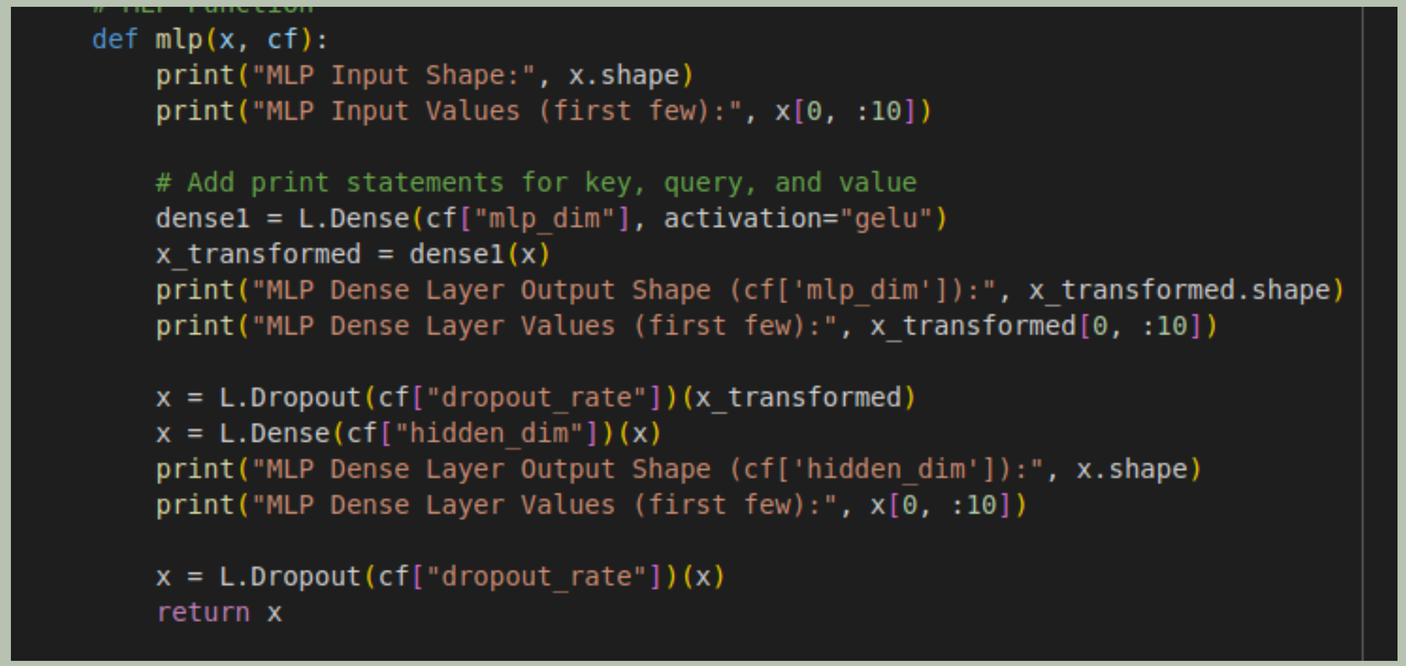
How MLP Works

The MLP block in the Transformer encoder enhances feature extraction with two fully connected layers and dropout for regularization.

1. Input:
    - The input tensor x has shape: (batch_size, num_patches, 128) (where 128 is the hidden_dim).
2. First Dense Layer:
    - The input passes through a Dense layer with 32 units.
    - GELU activation is applied, changing the tensor shape to: (batch_size, num_patches, 32).
3. Dropout:
    - Dropout with a rate of 0.1 is applied. The shape remains: (batch_size, num_patches, 32).
4. Second Dense Layer:
    - The output from the first Dropout layer passes through a Dense layer with 128 units.
    - Linear activation is applied, changing the tensor shape to: (batch_size, num_patches, 128).
5. Dropout Again:
    - Dropout with a rate of 0.1 is applied again. The shape remains: (batch_size, num_patches, 128).

    Output:

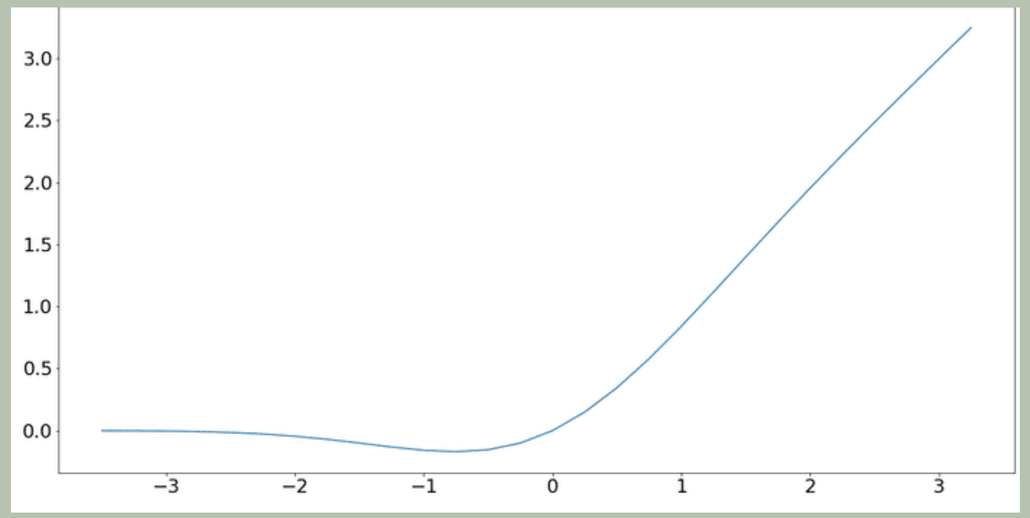
- The final output tensor has shape: (batch_size, num_patches, 128). This is the result after both Dense layers, activation, and dropout.



```
def mlp(x, cf):
    print("MLP Input Shape:", x.shape)
    print("MLP Input Values (first few):", x[0, :10])

    # Add print statements for key, query, and value
    dense1 = L.Dense(cf["mlp_dim"], activation="gelu")
    x_transformed = dense1(x)
    print("MLP Dense Layer Output Shape (cf['mlp_dim']):", x_transformed.shape)
    print("MLP Dense Layer Values (first few):", x_transformed[0, :10])

    x = L.Dropout(cf["dropout_rate"])(x_transformed)
    x = L.Dense(cf["hidden_dim"])(x)
    print("MLP Dense Layer Output Shape (cf['hidden_dim']):", x.shape)
    print("MLP Dense Layer Values (first few):", x[0, :10])

    x = L.Dropout(cf["dropout_rate"])(x)
    return x
```

GELU Activation function

$$\begin{aligned}\mathrm{GELU}(x) &= xP(X \leq x) = x\Phi(x) \\ &\approx 0.5x\left(1 + \tanh\left[\sqrt{2/\pi}\left(x + 0.044715x^3\right)\right]\right)\end{aligned}$$



- Gaussian Error Linear Unit is an activation function used in neural networks.
- It applies a smooth, continuous transformation to inputs, unlike ReLU which has a sharp cutoff.

Why GELU?

1. Smooth Activation: No abrupt cutoffs like ReLU.
2. Better Gradient Flow: Helps avoid "dead neurons" and allows stable training.
3. Probabilistic: Activates based on a smooth probability, especially for negative values.

Use Cases:

- Common in Transformers, like BERT and UNETR for tasks like image segmentation or text processing.

$$\begin{aligned}\mathrm{GELU}(x) &= xP(X \leq x) = x\Phi(x) \\ &\approx 0.5x\left(1 + \tanh\left[\sqrt{2/\pi}\left(x + 0.044715x^3\right)\right]\right)\end{aligned}$$

- x: The input value.
- P(X<=x) : The probability that a random variable X from a standard normal distribution is less than or equal to x. This is equivalent to the cumulative distribution function Φ(x) of the standard normal distribution.
- Φ(x): Cumulative Distribution function calculates the area under the curve of the normal distribution from −∞ to x.
- The Approximation formula: The calculation of Φ(x) includes complex integrals and the error function. It replaces the more complex exact formula with simpler operations like tanh (hyperbolic tangent) and a cubic term. This approximation achieves similar results but is faster for deep learning applications.
- GELU aligns well with global reasoning and attention mechanisms (used in Transformers).
- ReLU is optimal for localized spatial feature extraction (used in convolutional blocks).

MODEL SUMMARY

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 262144, 3)	0	-
dense (Dense)	(None, 262144, 64)	256	input_layer[0][0]
add (Add)	(None, 262144, 64)	0	dense[0][0]
layer_normalization (LayerNormalization)	(None, 262144, 64)	128	add[0][0]
multi_head_attention (MultiHeadAttention)	(None, 262144, 64)	99,520	layer_normalization[0… layer_normalization[0…
add_1 (Add)	(None, 262144, 64)	0	multi_head_attention[… add[0][0]
layer_normalization_1 (LayerNormalization)	(None, 262144, 64)	128	add_1[0][0]
dense_1 (Dense)	(None, 262144, 128)	8,320	layer_normalization_1…
dropout_1 (Dropout)	(None, 262144, 128)	0	dense_1[0][0]
dense_2 (Dense)	(None, 262144, 64)	8,256	dropout_1[0][0]
dropout_2 (Dropout)	(None, 262144, 64)	0	dense_2[0][0]
add_2 (Add)	(None, 262144, 64)	0	dropout_2[0][0], add_1[0][0]
layer_normalization_2 (LayerNormalization)	(None, 262144, 64)	128	add_2[0][0]
multi_head_attention_1 (MultiHeadAttention)	(None, 262144, 64)	99,520	layer_normalization_2… layer_normalization_2…
add_3 (Add)	(None, 262144, 64)	0	multi_head_attention_… add_2[0][0]
layer_normalization_3 (LayerNormalization)	(None, 262144, 64)	128	add_3[0][0]
dense_3 (Dense)	(None, 262144, 128)	8,320	layer_normalization_3…
dropout_4 (Dropout)	(None, 262144, 128)	0	dense_3[0][0]
dense_4 (Dense)	(None, 262144, 64)	8,256	dropout_4[0][0]
dropout_5 (Dropout)	(None, 262144, 64)	0	dense_4[0][0]
add_4 (Add)	(None, 262144, 64)	0	dropout_5[0][0], add_3[0][0]
layer_normalization_4 (LayerNormalization)	(None, 262144, 64)	128	add_4[0][0]
multi_head_attention_2 (MultiHeadAttention)	(None, 262144, 64)	99,520	layer_normalization_4… layer_normalization_4…
add_5 (Add)	(None, 262144, 64)	0	multi_head_attention_… add_4[0][0]
layer_normalization_5 (LayerNormalization)	(None, 262144, 64)	128	add_5[0][0]
dense_5 (Dense)	(None, 262144, 128)	8,320	layer_normalization_5…
dropout_7 (Dropout)	(None, 262144, 128)	0	dense_5[0][0]
dense_6 (Dense)	(None, 262144, 64)	8,256	dropout_7[0][0]
dropout_8 (Dropout)	(None, 262144, 64)	0	dense_6[0][0]

conv2d_transpose_8 (Conv2DTranspose)	(None, 256, 256, 32)	4,128	re_lu_8[0][0]
batch_normalization_6 (BatchNormalization)	(None, 128, 128, 64)	256	conv2d_6[0][0]
conv2d_9 (Conv2D)	(None, 256, 256, 32)	9,248	conv2d_transpose_8[0]…
re_lu_6 (ReLU)	(None, 128, 128, 64)	0	batch_normalization_6…
batch_normalization_9 (BatchNormalization)	(None, 256, 256, 32)	128	conv2d_9[0][0]
conv2d_transpose_5 (Conv2DTranspose)	(None, 256, 256, 32)	8,224	re_lu_6[0][0]
re_lu_9 (ReLU)	(None, 256, 256, 32)	0	batch_normalization_9…
concatenate_2 (Concatenate)	(None, 256, 256, 64)	0	conv2d_transpose_5[0]… re_lu_9[0][0]
conv2d_10 (Conv2D)	(None, 256, 256, 32)	18,464	concatenate_2[0][0]
reshape (Reshape)	(None, 512, 512, 3)	0	input_layer[0][0]
batch_normalization_10 (BatchNormalization)	(None, 256, 256, 32)	128	conv2d_10[0][0]
conv2d_12 (Conv2D)	(None, 512, 512, 16)	448	reshape[0][0]
re_lu_10 (ReLU)	(None, 256, 256, 32)	0	batch_normalization_1…
batch_normalization_12 (BatchNormalization)	(None, 512, 512, 16)	64	conv2d_12[0][0]
conv2d_11 (Conv2D)	(None, 256, 256, 32)	9,248	re_lu_10[0][0]
re_lu_12 (ReLU)	(None, 512, 512, 16)	0	batch_normalization_1…

batch_normalization_11 (BatchNormalization)	(None, 256, 256, 32)	128	conv2d_11[0][0]
conv2d_13 (Conv2D)	(None, 512, 512, 16)	2,320	re_lu_12[0][0]
re_lu_11 (ReLU)	(None, 256, 256, 32)	0	batch_normalization_1…
batch_normalization_13 (BatchNormalization)	(None, 512, 512, 16)	64	conv2d_13[0][0]
conv2d_transpose_9 (Conv2DTranspose)	(None, 512, 512, 16)	2,064	re_lu_11[0][0]
re_lu_13 (ReLU)	(None, 512, 512, 16)	0	batch_normalization_1…
concatenate_3 (Concatenate)	(None, 512, 512, 32)	0	conv2d_transpose_9[0]… re_lu_13[0][0]
conv2d_14 (Conv2D)	(None, 512, 512, 16)	4,624	concatenate_3[0][0]
batch_normalization_14 (BatchNormalization)	(None, 512, 512, 16)	64	conv2d_14[0][0]
re_lu_14 (ReLU)	(None, 512, 512, 16)	0	batch_normalization_1…
conv2d_15 (Conv2D)	(None, 512, 512, 16)	2,320	re_lu_14[0][0]
batch_normalization_15 (BatchNormalization)	(None, 512, 512, 16)	64	conv2d_15[0][0]
re_lu_15 (ReLU)	(None, 512, 512, 16)	0	batch_normalization_1…
conv2d_16 (Conv2D)	(None, 512, 512, 1)	17	re_lu_15[0][0]

Total params: 2,398,193 (9.15 MB)
Trainable params: 2,396,465 (9.14 MB)
Non-trainable params: 1,728 (6.75 KB)

1. Dice Coefficient

Formula:

$$\text{Dice Coefficient} = \frac{2 \cdot |A \cap B| + \text{smooth}}{|A| + |B| + \text{smooth}}$$

2. Dice Loss

Formula:

$$\text{Dice Loss} = 1 - \text{Dice Coefficient}$$

3. Precision

Formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

4. Sensitivity (Recall)

Formula:

$$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

5. Specificity

Formula:

$$\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

6. Intersection over Union (IoU)

Formula:

$$\text{IoU} = \frac{|A \cap B| + \text{smooth}}{|A \cup B| + \text{smooth}}$$

7. Weighted Dice Loss

Formula:

$$\text{Weighted Dice Loss} = 1 - \frac{2 \cdot \text{Weighted Intersection}}{\text{Weighted Sum of True and Predicted}}$$

8. Tversky Loss

Formula:

$$\text{Tversky Index} = \frac{\text{True Positives} + \text{smooth}}{\text{True Positives} + \alpha \cdot \text{False Negatives} + \beta \cdot \text{False Positives} + \text{smooth}}$$

$$\text{Tversky Loss} = 1 - \text{Tversky Index}$$

9. Focal Dice Loss

Formula:

$$\text{Focal Loss} = -\alpha(1 - p_t)^\gamma \log(p_t)$$

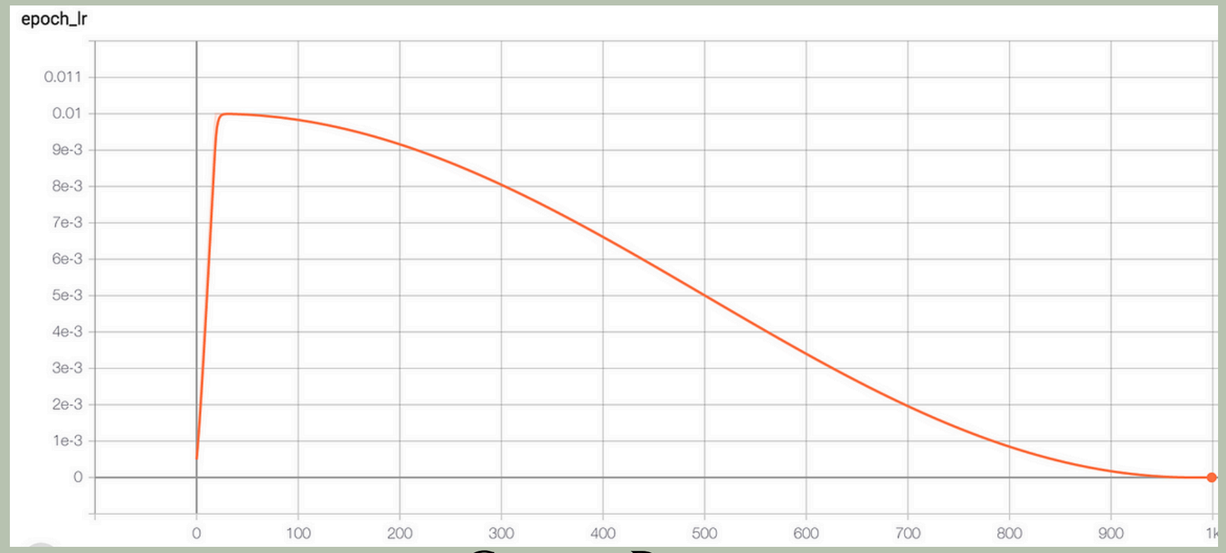
$$\text{Focal Dice Loss} = \text{Focal Loss} + \text{Dice Loss}$$

- **Dice Coefficient vs IoU**: Both measure overlap; Dice is more sensitive to smaller objects.
- **Precision vs Recall**: High precision reduces false positives, while high recall reduces false negatives.
- **Tversky vs Dice**: Tversky is a weighted form of Dice, offering more control over false positives/negatives.
- **Focal Dice vs Weighted Dice**: Both address imbalance, but Focal Dice emphasizes hard-to-predict samples.

Learning Rate Decay

$$\text{LearningRate}(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\max}}\pi\right)\right)$$

```
initial_lr = 1e-4
decay_steps = num_epochs * len(train_dataset)
lr_schedule = CosineDecay(
    initial_learning_rate=initial_lr,
    decay_steps=decay_steps,
    alpha=0.0
)
```



Cosine Decay

- Gaussian Error Linear Unit is an activation functio used in neural networks.
- It applies a smooth, continuous transformation to inputs, unlike ReLU which has a sharp cutoff.

Why GELU?

1. Smooth Activation: No abrupt cutoffs like ReLU.
2. Better Gradient Flow: Helps avoid "dead neurons" and allows stable training.
3. Probabilistic: Activates based on a smooth probability, especially for negative values.

Use Cases:

- Common in Transformers, like BERT and UNETR for tasks like image segmentation or text processing.

`dice_coef: 0.6509` Measures overlap between predicted and true regions, emphasizing both precision and recall (ranges from 0 to 1, where 1 is perfect overlap).

`iou: 0.5256` (Intersection over Union)*: Ratio of the overlap area to the combined area of predicted and true regions, indicating how well they align.

`precision: 0.6596` Proportion of correctly predicted positive pixels out of all pixels predicted as positive (focuses on minimizing false positives).

`acc: 0.9880` Proportion of correctly classified pixels (both positive and negative) out of all pixels.

`loss: 0.3163` Calculated as 1 -text{metric} (e.g., (1 - text{Dice}) or (1 - text{IoU})) to penalize the model for poor segmentation during training.

3:38 0.54 KB/S 20%

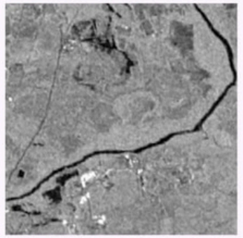
Flood Area Detection

Upload an image to detect flood risks.

Upload Image

Detect Flood

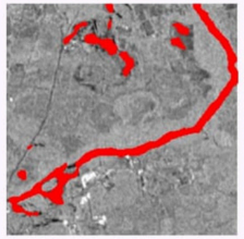
Uploaded Image



Ground Truth Image

Predicted Mask

Flood Detected Image



Clear Image

Flood Area Detection

Detect Flood

Ground Truth Image

Predicted Image

Flood Detected Image